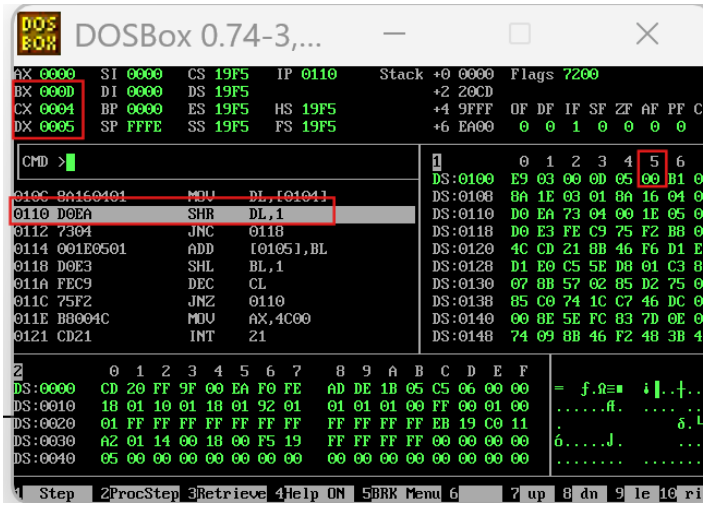
CODE:

```
1   [org 0x100]
2                   jmp   start
3
4   multiplicand:   db    13
5   multiplier:     db    5
6
7   result:         db    0
8
9   start:          mov   cl, 4
10                  mov   bl, [multiplicand]
11                  mov   dl, [multiplier]
12
13  checkbit:       shr   dl, 1
14                  jnc   skip
15                  add   [result], bl
16
17  skip:           shl   bl, 1
18                  dec   cl
19                  jnz   checkbit
20
21                  mov   ax, 0x4c00
22                  int   0x21
```



Why shifting right?

To check if DX(multiplier) has 0 or 1 in its

LSB :

- If its 0 no operation to be performed only

shifting required

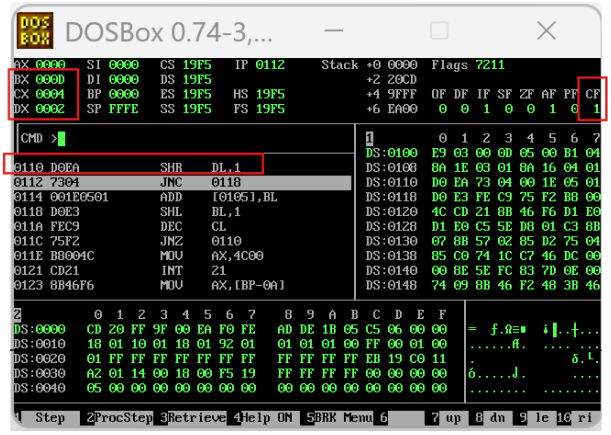
- If its 1 add multiplier to the result and then

do the shift

Before shifting :

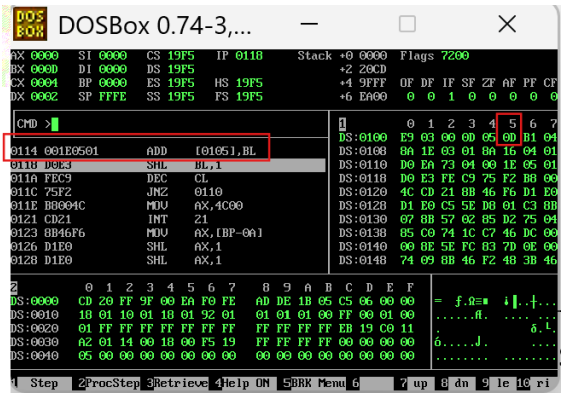
- BX stores 000D (1101 in binary).
- CX stores the no of iteration required for multiplication(max number of bits)
- DX stores the multiplier 5 (0101).
- The memory location result (0105) stores value of result which is 0 initially (this will change when we add into result)

After the first shift:



- DX has changed from : **0101(5)->0010(2)** {%2}
- Least significant bit before rotation **sent to CF**

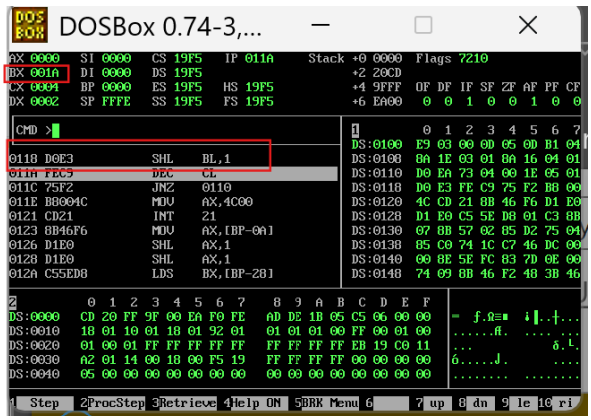
Now as **CX flag has been raised** the result 0010(2) will be added to the result so it would **execute the next instruction(ADD)** and **not jump**:



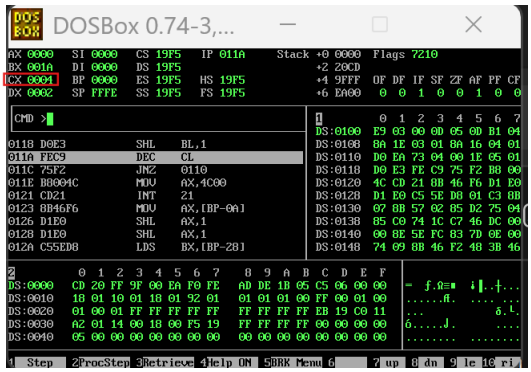
The memory location 0105 now stores 0D:

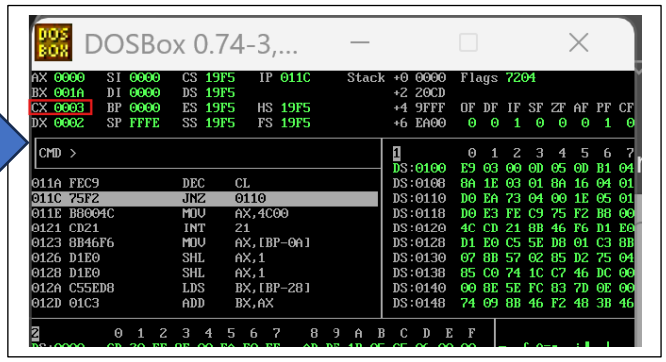
- Value of multiplicand has been added into result as CF was raised

SHL will shift 1 bit left the value of bx which changes from **0000 1101(13)->0001 1010(A):**

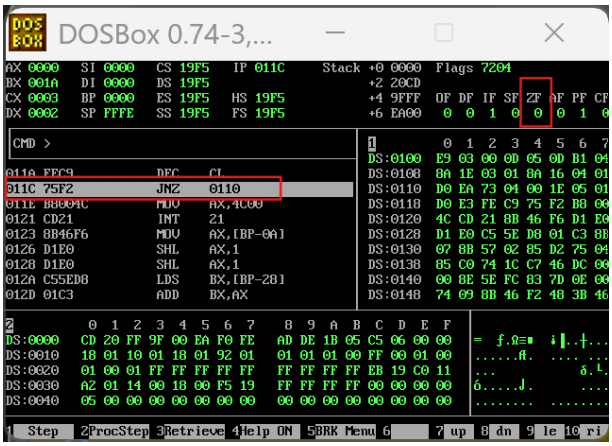


After this step cx will be dec by 1 as 1 iteration has been completed and result is added:



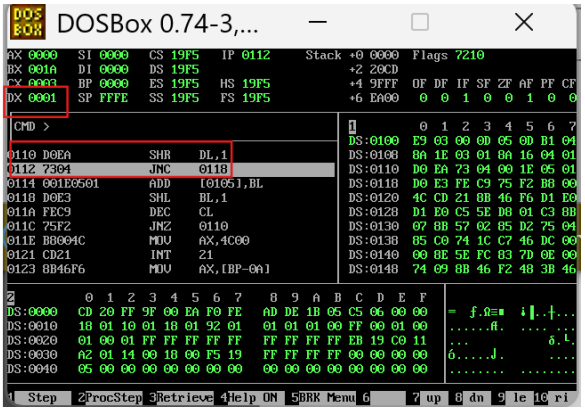


Now jnz will check if whole bits of multiplier have been processed or not which is indicated by 0 flag :



As zero flag is not raised it means that some iteration is Left so do the whole process again until ZF is raised

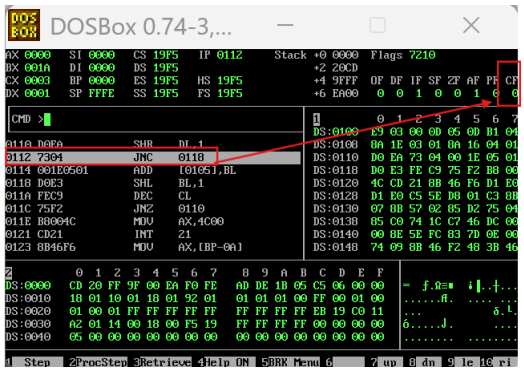
2ND ITERATION: NOW WE WILL PROCESS **SECOND MSB** OF MULTIPLIER 0101:

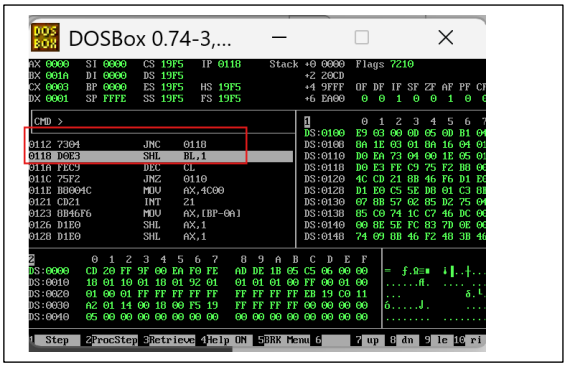


Shift right operation has been performed which changes DX from **0010(2)->0001(1) %2.**

And 2nd MSB of 5 now MSB will be sent to CF so CF Remain 1.

Now we will check if 2nd MSB is 0 or 1 as 2nd MSB of 5(0101) is 0 **carry would not be raised** because of which the **step of adding bits into result would be skipped** and **program directly would jump to shift left:**

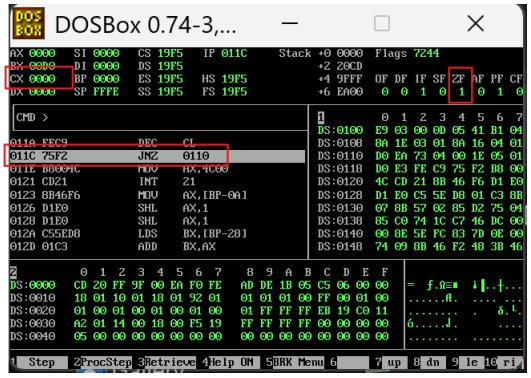




From above we see no addition command has been executed.

And the memory location 0105 still stores the same result as before (OD)

Similarly next 2 iterations are performed so as it is already demostrated I will now skip to show how program ends:



When cx becomes 0 :

- 0 flag is raised
- Due to which no jump is performed
- Program is terminated.